

Phase-based predicting the battery remaining time for Android mobile devices

Seung-Ryeol Ohk
Department of Electrical and Computer
Engineering
Ajou University
Suwon, Korea
dhrtmdfuf@ajou.ac.kr

Seok Min Hong
Department of Electrical and Computer
Engineering
Ajou University
Suwon, Korea
smhong1123@ajou.ac.kr

Young-Jin Kim
Department of Electrical and Computer
Engineering
Ajou University
Suwon, Korea
youngkim@ajou.ac.kr

Abstract— Since smartphones are battery-driven, it is important to know how power-efficiently and how long they are used in a limited battery capacity. As smartphones become popular and their usage time increases, many low-power techniques have been studied to use smartphones' batteries efficiently. However, most low-power techniques are only interested in techniques that reduce power purely, and do not care about the feasible time of the current workload based on the battery remaining time. In this paper, we propose a method to predict the accurate power consumption of recent smartphones for development of efficient low power techniques. Our method estimates the battery remaining time to predict the viable time of the running workload with the remaining smartphone battery. In detail, it predicts the overall power consumption for a smartphone by using power models for a CPU, a GPU, and a display that have a high proportion of energy consumption in smartphones. Through experiments, we build a prediction model that calculates battery consumption by using the predicted power consumption according to CPU/GPU phase of the workload used in the experiment. Consequently, we complete a technique to predict battery consumption and remaining time for arbitrary workloads.

Keywords—mobile device, Android, battery remaining time, phase, power estimation, smartphone

I. INTRODUCTION

We are exposed to a lot of mobile devices in our daily lives. Some devices use power cables, while others use batteries. Among them, a representative mobile device is a smartphone. People want smartphones to be able to use less batteries for a long time. Therefore, power management of smartphones is considered as an important technology of smartphones and has been developed. In order to manage the power of a smartphone, it is necessary to know the major power consumption of a smartphone. Most of the components that consume power in smartphones are CPUs, GPUs, and displays. Therefore, it is important to predict the power consumption of CPUs, GPUs, and displays accurately. Existing low-power techniques [1][2][3] have mostly been interested in reducing power consumption without calculating it.

In this paper, we use the power models [4][5] to predict the power consumption of CPUs, GPUs, and displays. Power models were built and validated using Monsoon Power Monitors [6]. We then calculate battery consumption with the predicted power consumption to estimate the battery remaining time of a smartphone. Depending on the workload during the experiment, different CPU/GPU phases of a smartphone are determined and different estimation parameters are calculated. Consequently, the corrected parameters are calculated by dividing the cases according to the CPU/GPU phase. This paper proposes a method to predict the power consumption of smartphones accurately and the battery consumption time according to the battery remaining

time. Predicting power consumption of smartphones by employing power models of a CPU, a GPU, and a display and calculating accurate battery consumption according to CPU/GPU phases implements our battery remaining time estimation method.

The remainder of this paper is organized as follows. In Section II, we explore the need to predict battery remaining time according to the way existing low-power techniques are implemented, which motivates this study. Section III describes how to predict smartphone power consumption and calculate calibrated battery consumption according to CPU/GPU phases in order to estimate the battery consumption time according to smartphone power consumption and its remaining battery capacity. Section IV explains the experimental environment and experimental results. Finally, Section V concludes this paper.

II. BACKGROUND

Existing low-power techniques employ Dynamic Voltage and Frequency Scaling (DVFS) in a way that only reduces power consumption [1][2][3]. This is not a problem in implementing low-power techniques, but it is not possible to know how long the running workload in the changed environment can consume the battery. From the smartphone user's perspective, it can be very good information on how much work is currently available to run with the remaining battery. Therefore, we devise a method to estimate the viable time of a running workload based on the phases of CPU and GPU using power models of CPU, GPU, and display.

There is a study using CPU/GPU utilization to estimate battery remaining time [7]. It used a power model to predict power consumption and classified workloads into CPU/GPU utilization sizes to predict battery time remaining. However, workload classification using CPU/GPU utilization only considers processor usage and cannot distinguish between operational states. To compensate for this limitation, this paper classifies the hardware behavioral state of the workload, names it phase, and predicts the battery remaining time according to phase.

III. PROPOSED METHOD

CPU and GPU power consumption prediction uses a power model [4]. It predicts power consumption by reading the performance monitoring units (PMUs) of the CPU and GPU. CPU power consumption prediction is P_{CPU} and GPU power consumption prediction is P_{GPU} . The display power consumption prediction employs a power model [5]. Power consumption is predicted using display pixel values and precalculated lookup tables (LUTs). The estimated display power consumption are $P_{display}$. CPU, GPU, and display power consumption predictions are performed at an interval of 50 ms to predict power consumed over 50 ms.

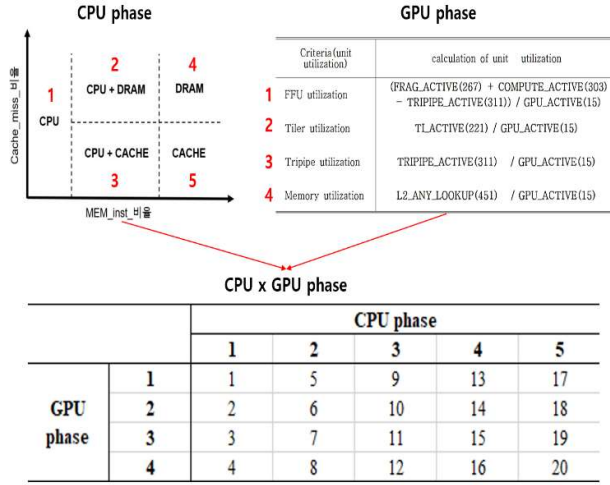


Fig. 1. Combination of CPU and GPU phases

In this paper, phase means the execution state in which CPU/GPU is occupied by the workload. CPU phase is classified according to how many and how much memory the CPU uses by analyzing the instructions it performs. It classifies CPU phases based on the cache miss ratio and LDST instruction ratio of the CPU with a total of five CPU phases. GPU phase is classified by utilization of the units that make up the GPU.

The GPU is divided into fixed function units, tiler units, tripipe units, and memory units, and the phase corresponding to the maximum value of utilization of each unit is selected as the GPU phase. There are a total of four GPU phases. A combination of CPU and GPU phases produced a total of 20 phases, as shown in Figure 1.

When CPU, GPU, and display power consumption calculated above are named as P_{CPU} , P_{GPU} , and $P_{display}$, respectively, battery consumption $|\Delta E|$ is calculated as shown in Eq. (1).

$$|\Delta E| = \alpha \times (P_{CPU} + P_{GPU} + P_{display}) \times t \quad (1)$$

α is the coefficient and t is the measurement time of power consumption multiplied by the energy of the battery to match the unit. In order to check the difference in battery consumption ratio depending on the remaining battery, we compared the calculation when the battery remains at 70% and 30%. The result showed a negligible difference of about 1%.

CPU/GPU phase share varies depending on workload. We measured power consumption and battery consumption by classifying workload intervals with specific phases. We then calculated the constant α required to calculate the battery remaining time by phase. Table I shows 20 CPU/GPU phase-specific constant α .

We use workloads that are not used for α calculation to predict battery remaining time. After running the workload, CPU/GPU phases are collected in real time with a single sample per 50ms, and when the samples are collected, they are averaged to classify the CPU/GPU phase status of the current workload.

Based on the CPU/GPU phase status classified, the adjusted predicted consumption battery is calculated using the coefficients calculated in Table I. If the predicted battery consumption value is divided by 50 ms, we can obtain it per

second. Now, dividing the remaining battery by the battery consumption prediction per second can calculate the battery remaining time prediction as shown in Eq. (2).

$$T_{est}(s) = \frac{battery(E)}{battery_{est}(E/s)} \quad (2)$$

T_{est} is the predicted battery reaming time, $battery$ is the remaining battery, and $battery_{est}$ is the predicted battery consumption per second.

IV. EXPERIMENTAL RESULTS

The smartphone used in the experiment is the Samsung Galaxy S7. The Samsung Galaxy S7 smartphone uses the Android operating system based on the Linux kernel. The processor uses an Exynos 8 Octa (8890) SoC as shown in Table II. The processor is powered by Samsung Exynos M1 MP4 2.29 GHz + ARM Cortex-A53 MP4 1.59 GHz as CPU and ARM Mali-T880 MP12650 MHz as GPU. The CPU is an octa-core CPU with Samsung Exynos M1 as a big cluster with quad-core configurations and ARM Cortex-A53 as a little cluster with quad-core configurations that is applied with the ARM big.LITTLE solution.

TABLE I. COEFFICIENT ALPHA CLASSIFIED BY CPU/GPU PHASES

α		CPU phase				
		1	2	3	4	5
GPU phase	1	0.000179	0.000175	0.000160	0.000161	0.000140
	2	0.000192	0.000188	0.000172	0.000173	0.000150
	3	0.000195	0.000189	0.000176	0.000175	0.000151
	4	0.000185	0.000179	0.000167	0.000166	0.000143

TABLE II. CPU AND GPU ON SAMSUNG GALAXY S7

Processor of Samsung Galaxy S7	
CPU	Samsung Exynos M1 MP4 2.29 GHz + ARM Cortex-A53 MP4 1.59 GHz
GPU	RM Mali-T880 MP12 650 MHz

TABLE III. 3D GAME WORKLOADS FOR EXPERIMENT

No	workload
1	3DMark SLING SHOT EXTREME
2	3DMark SLING SHOT
3	3DMark ICE STORM EXTREME
4	3DMark ICE STORM
5	3DMark API OVERHEAD
6	Madagascar 3D Benchmark
7	Benchmark 3D Kassja graphics
8	Renderscript Benchmark
9	NenaMark2
10	V1 - GPU Benchmark Pro
11	Seascape Benchmark
12	Basemark GPU
13	3D Benchmark Android Gamers
14	Relative Benchmark
15	L-bench

A total of 15 workloads were installed on the Samsung Galaxy S7 smartphone in the Google Play Store. The workloads installed are 3DMark, Madagascar 3D benchmark, Kassja benchmark, Renderscript benchmark, Nena benchmark, V1 GPU benchmark, Seascape benchmark, Basemark, 3D benchmark, Relative benchmark, and L-bench, as shown in Table III. The 3DMark workload are treated as five different workloads in terms of section. In this way, they were used to verify the performance of battery remaining time prediction.

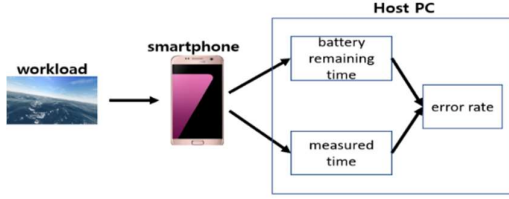


Fig. 2. Experimental environment

Calculating CPU and GPU power consumption starts with counting the events in CPU and GPU using performance monitoring counter (PMC) used by a gator driver of the Linux kernel. CPU and GPU power models [4] calculate the power consumption by multiplying the number of events by the factor made from the genetic algorithm. Because this calculation has real and logarithmic operations, it cannot be computed on the Linux kernel that uses only integers. Thus, a native app is employed, which computes real and logarithmic operations, calculates the power consumption, and sends the result to the Linux kernel. The power consumption of the display is calculated using the display power model [5]. For the same reason as CPU and GPU power models, the power consumption is calculated using a native app and sent to the Linux kernel. Then, the FPS is calculated by counting the frames composed each second on SurfaceFlinger [7] that composes the display frame. The calculated power consumption and FPS are sent via sysfs [8] to the Linux kernel. To share data divided across multiple drivers in the Linux kernel, we use the EXPORT_SYMBOL [9] command.

Figure 2 is the experimental environment that shows the error rate by running the workload on a smartphone and comparing the measured time with the predicted battery remaining time. To predict battery remaining time, we execute one of 15 workloads on the smartphone. Then, we predict the phases of CPU and GPU and the power consumption of smartphones. We estimate the phase and power consumption at an interval of 50 ms using the power model. At this interval, battery remaining time is predicted using the phase and power consumption, and a pre-calculated coefficient alpha table is used to predict the battery remaining time. We also measure the time of running the workload while predicting the battery remaining time by running the workload on the smartphones. Then, we compare the time spent with the predicted battery remaining time.

We used the coefficient alpha for battery remaining time prediction to predict the battery remaining time. We compare the battery remaining time prediction results using the CPU/GPU utilization-based method [10] with the battery remaining time prediction results using the CPU/GPU phase-based method proposed in this paper. The results of each battery remaining time prediction were compared using the error rate of the measured battery run-down time.

Table IV shows the error rates of the battery remaining time prediction using CPU/GPU utilization and the proposed battery remaining time prediction using CPU/GPU phase over the real battery remaining time. The proposed method based on phase yields 2% more accurate results.

TABLE IV. BATTERY REMAINING TIME COMPARISON

	Utilization-based [7]	Phase-based
Error rate (%)	9	7

V. CONCLUSION

In this paper, we propose a method for estimating the accurate system power using CPU, GPU, and display power models, and predicting the phase-based battery remaining time by using the power estimation for Android mobile devices. Compared to previous studies, the battery remaining time prediction using phase showed a 2% more accurate result. This study helps smartphones and users recognize how long they can perform the desired tasks with their current batteries by calculating the battery remaining time, which is not present in existing smartphone low-power technology studies. As future work, combined with low-power techniques, a new low-power technology could be completed by considering display power, unlike the low-power techniques that only consider conventional CPU and GPU power. It can also be implemented on multiple Android devices.

ACKNOWLEDGMENT

This work has been supported by the Future Combat System Network Technology Research Center program of Defense Acquisition Program Administration and Agency for Defense Development.(UD190033ED)

REFERENCES

- [1] Jum-Gyu Park, Nikil Dutt, Hoyeonjiki Kim, Sung-Soo Lim, HiCAP: Hierarchical FSM-based Dynamic Integrated CPU-GPU Frequency Capping Governor for Energy-Efficient Mobile Gaming, ISLPED 2016, August 2016.
- [2] Anuj Pathania, Alexandru Eugen Irimiea, Alok Prakash, Tulika Mitra, Power-performance modelling of mobile gaming workloads on heterogeneous MPSoCs, DAC 2015, June 2015.
- [3] Jum-Gyu Park, Nikil Dutt, Hoyeonjiki Kim, Sung-Soo Lim, Co-Cap: energy-efficient cooperative CPU-GPU frequency capping for mobile games, SAC 16, April 2016.
- [4] Lee, K.; Ohk, S.-R.; Lim, S.-G.; Kim, Y.-J. Phase-Based Accurate Power Modeling for Mobile Application Processors. *Electronics* 2021, 10, 1197. <https://doi.org/10.3390/electronics10101197>
- [5] S. W. Hong, S. W. Kim and Y. J. Kim, "3 channel dependency-based power model for mobile AMOLED displays," in *Proceedings of the 54th Annual Design Automation Conference*, pp. 1-6, Jun. 2017.
- [6] Monsoon Solutions. Power Monitor [Online]. Available : <http://www.monsoon.com/LabEquipment/PowerMonitor/>
- [7] Ren, Chuangang, Peng Liu, and Sencun Zhu. "WindowGuard: Systematic Protection of GUI Security in Android." *NDSS*. 2017.
- [8] Mochel, Patrick. "The sysfs filesystem." *Linux Symposium*. 2005.
- [9] Zhang, Xiang-li, Xiao-qi Tang, and Ji-hong Chen. "Time synchronization of hierarchical real-time networked CNC system based on ethernet/internet." *The International Journal of Advanced Manufacturing Technology* 36.11 (2008): 1145-1156.
- [10] Seung-Ryeol Ohk, Young-Jin Kim. Predicting the battery remaining time using CPU-GPU-display power consumption prediction for smartphones. *Proc. of Korea Software Congress 2020*, pp.111-112, Dec. 2020.